



10-423/10-623 Generative AI

Machine Learning Department
School of Computer Science
Carnegie Mellon University

Generative Adversarial Networks (GANs) + Probabilistic Graphical Models

Aran Nayebi & Matt Gormley
Lecture 6

Reminders

- Homework 1: Generative Models of Text
 - Out: Mon, Jan 26
 - Slot A Due: Mon, Feb 9 at 11:59pm
- Remember, no AI use is permitted in Slot A! It will be considered an academic integrity violation. We want you to have the opportunity to think for yourself, so save the AI use for Slot B, when it's permitted!
- Regrade requests will only be for Slot B submissions, not Slot A (and only after Slot B is graded).
- If you got 100% on a question, you do not need to resubmit for Slot B (your score is the maximum of the two). In general, you do not need to submit anything to Slot B if you are happy with your Slot A grade!

TASK: IMAGE GENERATION

Image Generation

- Class-conditional generation
- Super resolution
- Image Editing
- Style transfer
- Text-to-image (TTI) generation



Figure from Razavi et al. (2019)

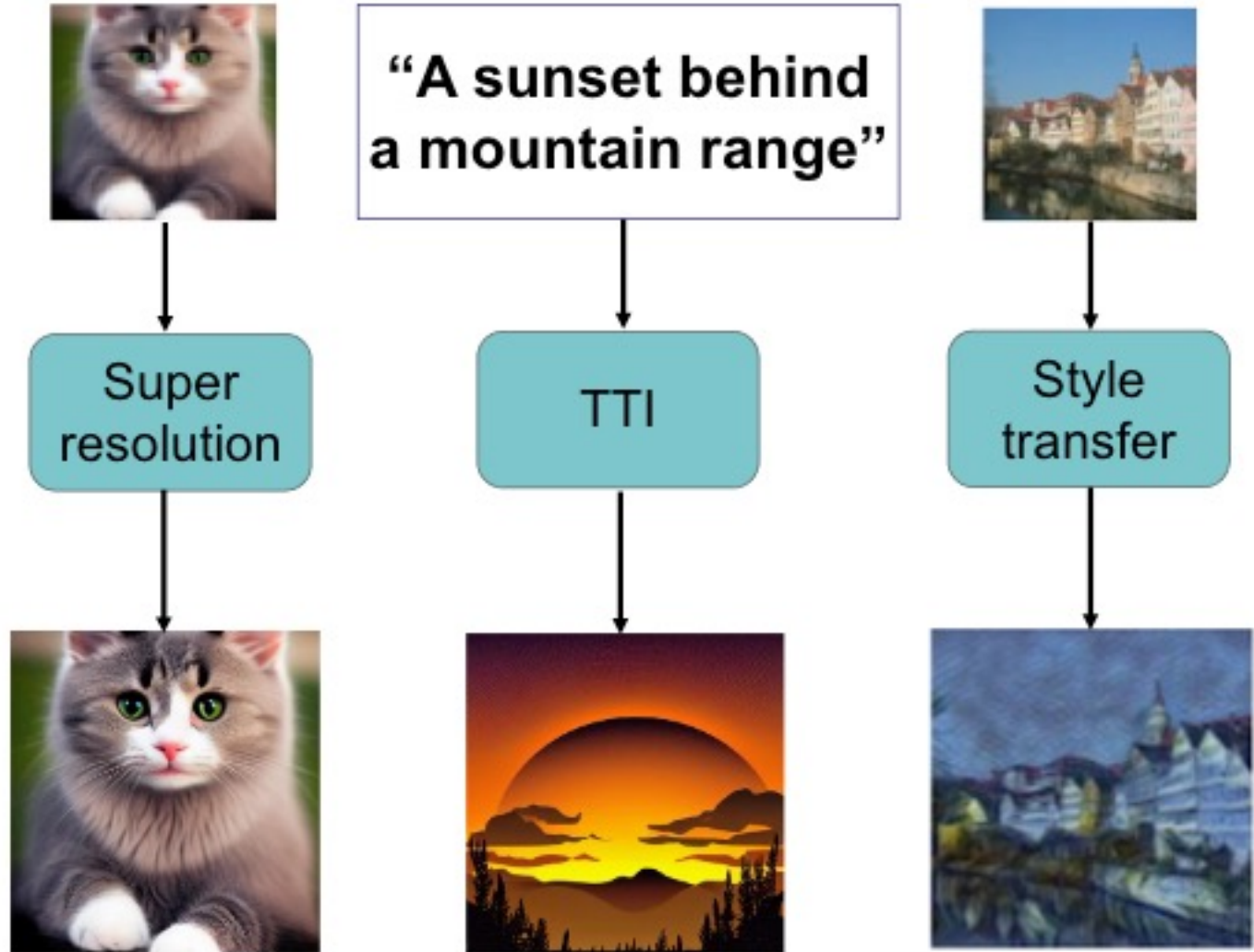


Figure from Bie et al. (2023)

Class Conditional Generation

- **Task:** Given a class label indicating the image type, sample a new image from the model with that type
- Image classification is the problem of taking in an image and predicting its label $p(y|x)$
- Class conditional generation is doing this in reverse $p(x|y)$

sea anemone

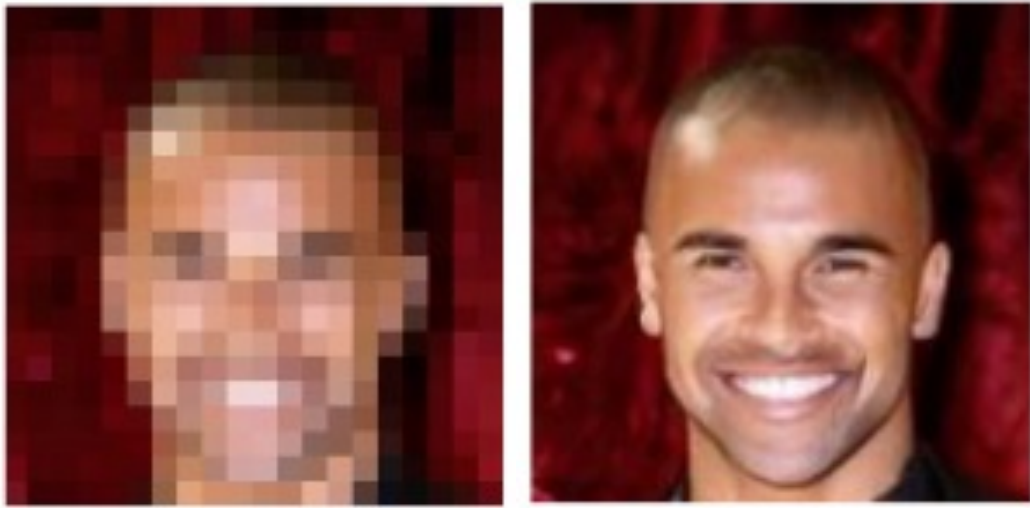
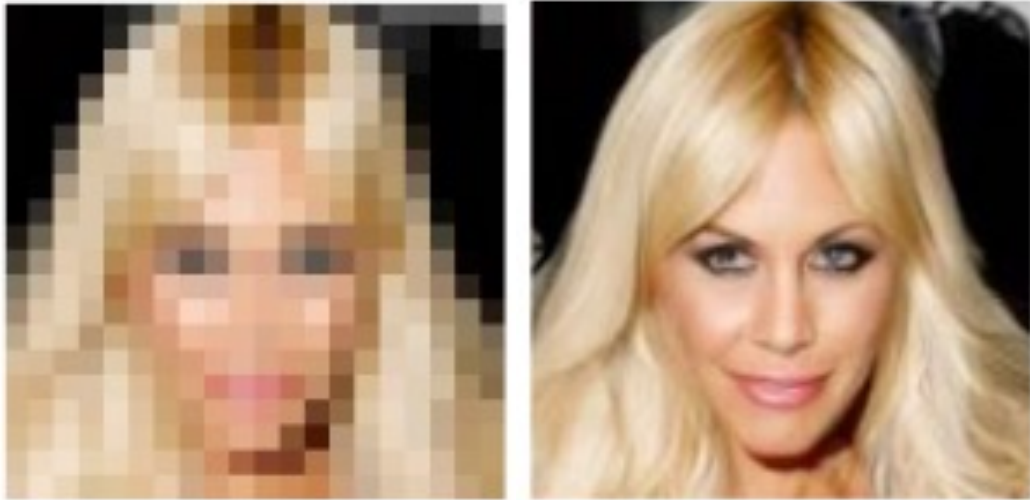
brain coral

slug

goldfinch



Super Resolution



LR

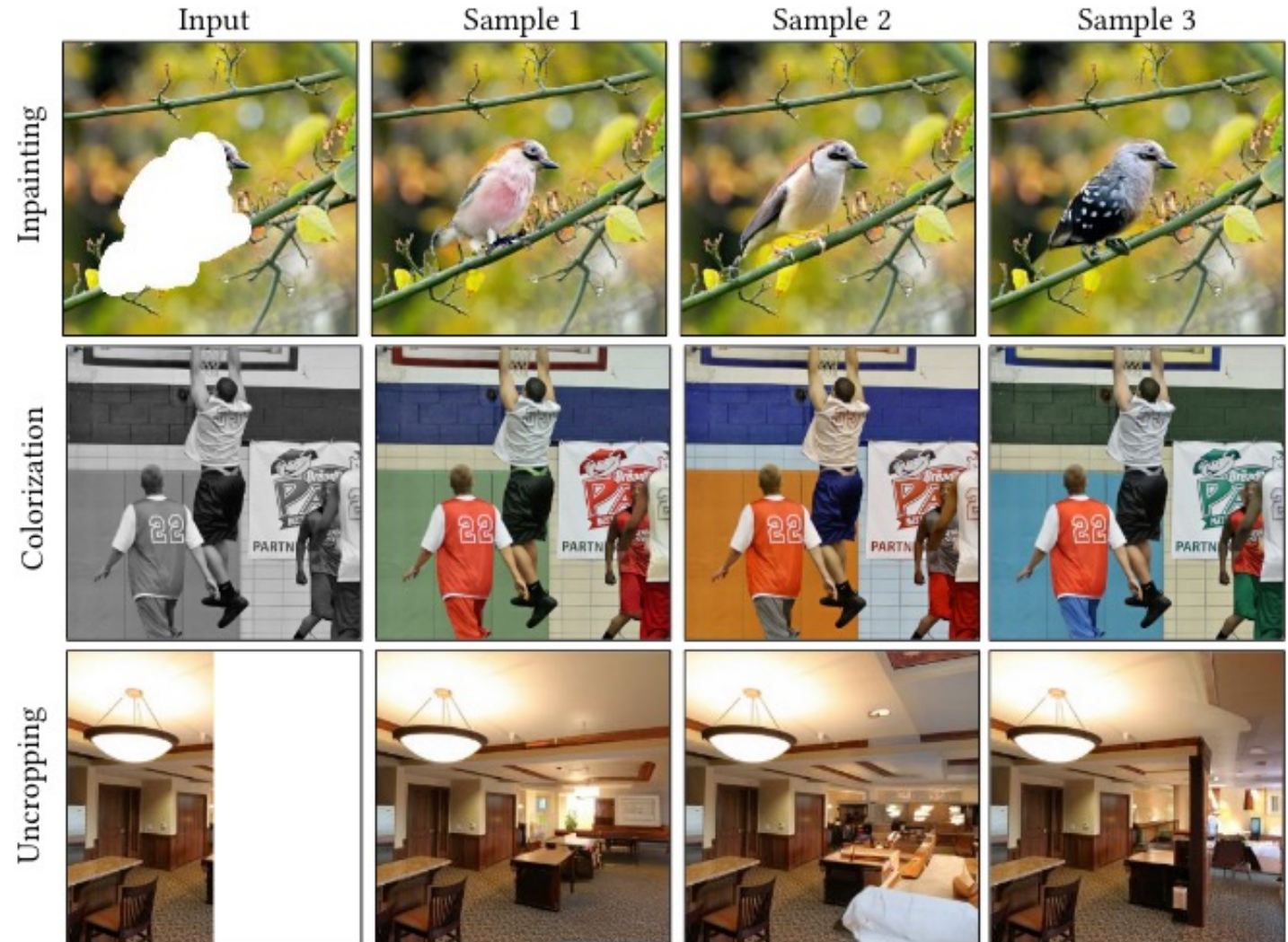
SRDiff

- Given a low resolution image, generate a high resolution reconstruction of the image
- Compelling on low resolution inputs (see example to the left) but also effective on high resolution inputs

Image Editing

A variety of tasks involve automatic editing of an image:

- **Inpainting** fills in the (pre-specified) missing pixels
- **Colorization** restores color to a greyscale image
- **Uncropping** creates a photo-realistic reconstruction of a missing side of an image



Style Transfer

- The goal of style transfer is to blend two images
- Yet, the blend should retain the semantic content of the source image presented in the style of another image



Figure 3. Images that combine the content of a photograph with the style of several well-known artworks. The images were created by finding an image that simultaneously matches the content representation of the photograph and the style representation of the artwork. The original photograph depicting the Neckarfront in Tübingen, Germany, is shown in **A** (Photo: Andreas Praefcke). The painting that provided the style for the respective generated image is shown in the bottom left corner of each panel. **B** *The Shipwreck of the Minotaur* by J.M.W. Turner, 1805. **C** *The Starry Night* by Vincent van Gogh, 1889. **D** *Der Schrei* by Edvard Munch, 1893. **E** *Femme nue assise* by Pablo Picasso, 1910. **F** *Composition VII* by Wassily Kandinsky, 1913.

Text-to-Image Generation

- Given a text description, sample an image that depicts the prompt
- From Google Gemini 2.5 Flash Image (Nano Banana)

Figure from: https://www.linkedin.com/posts/alka-gupta-ag_so-i-bumped-into-taylor-swift-at-changi-airport-activity-7369534745061322753-296P



Text-to-Image Generation

- Given a text description, sample an image that depicts the prompt
- The following images are samples from SDXL with refinement

Prompt: close up headshot, futuristic **old man**, wild hair sly smile in front of gigantic UFO, dslr, sharp focus, dynamic composition, **rule of thirds**



MODEL: GENERATIVE ADVERSARIAL NETWORK (GAN)

Stable Diffusion (Spring 2026) still can't explain GANs

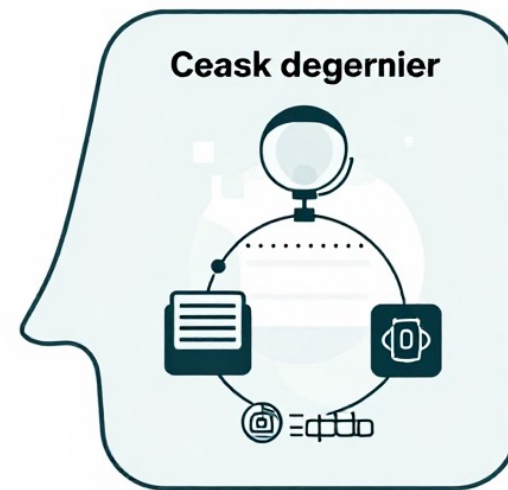
Prompt: slide explaining Generative Adversarial Networks (GANs) for a Machine Learning course, carefully designed, easy to follow

Negative Prompt: boring, unclear, nontechnical

Generative Adversarial Networks (GANs)

GANs): tie inderict. and peopicat maic encpem to Generative Adversarial knows nabecal: that induss the a cenracies in cacalion.

Aneido:
Emechpic ancogel
(Ncuption. The our trie).



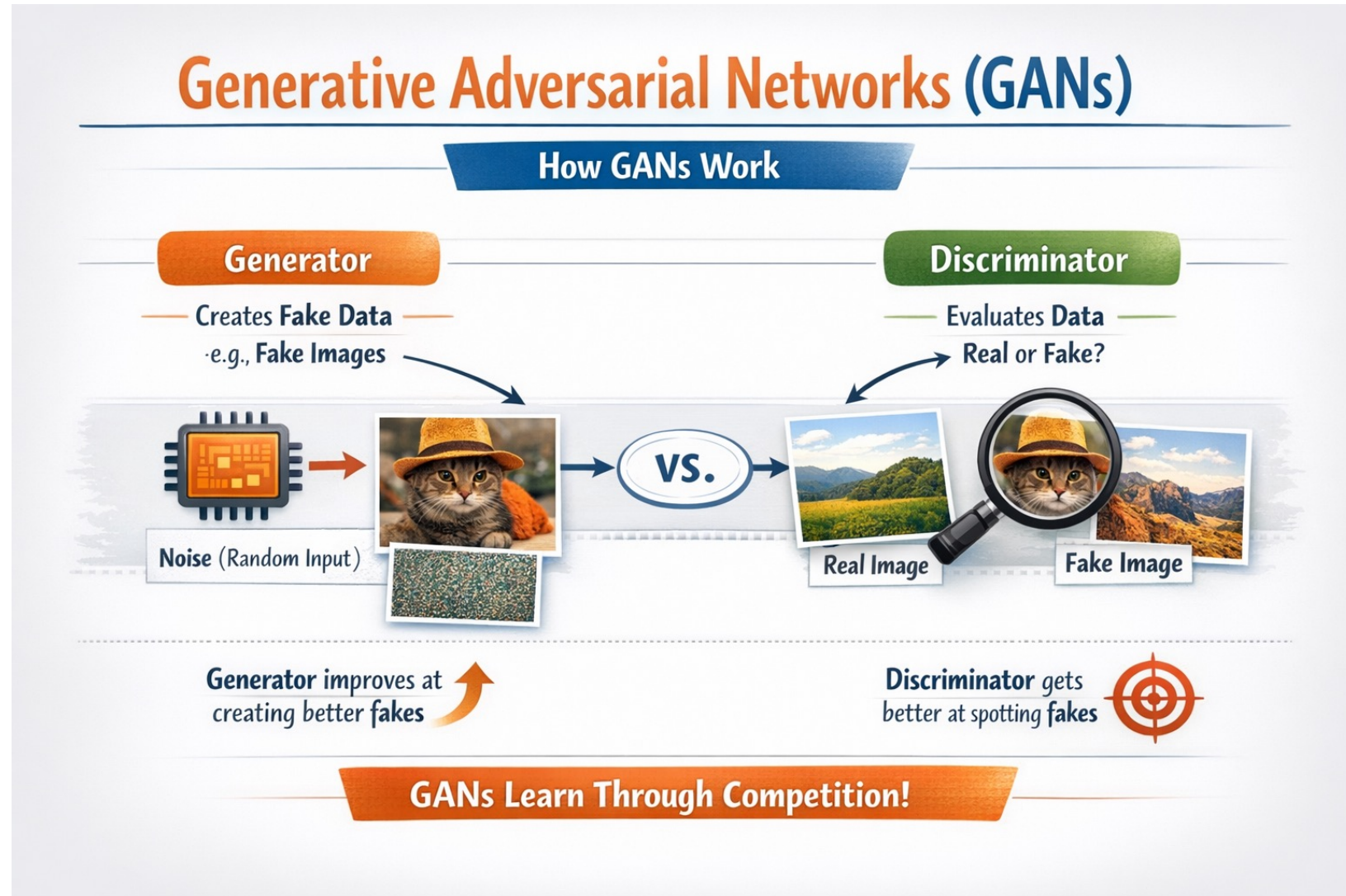
Machine Leaaring course
**Generative Adversarial
(GAwnik)**

Madiineergo ai derieminte

GPT5 (Spring 2026) does a much better (but not detailed!) job at explaining GANs

Prompt: Generate a slide (as an image) explaining Generative Adversarial Networks (GANs) for a Machine Learning course, carefully designed, easy to follow

Negative Prompt: boring, unclear, nontechnical



Generative Adversarial Networks (GANs)

A GAN consists of two deterministic neural network models:

1) the Generator

takes a vector of random noise as input,
and generates an image

2) the Discriminator

takes in an image classifies whether it is
real (label 1) or fake (label 0)

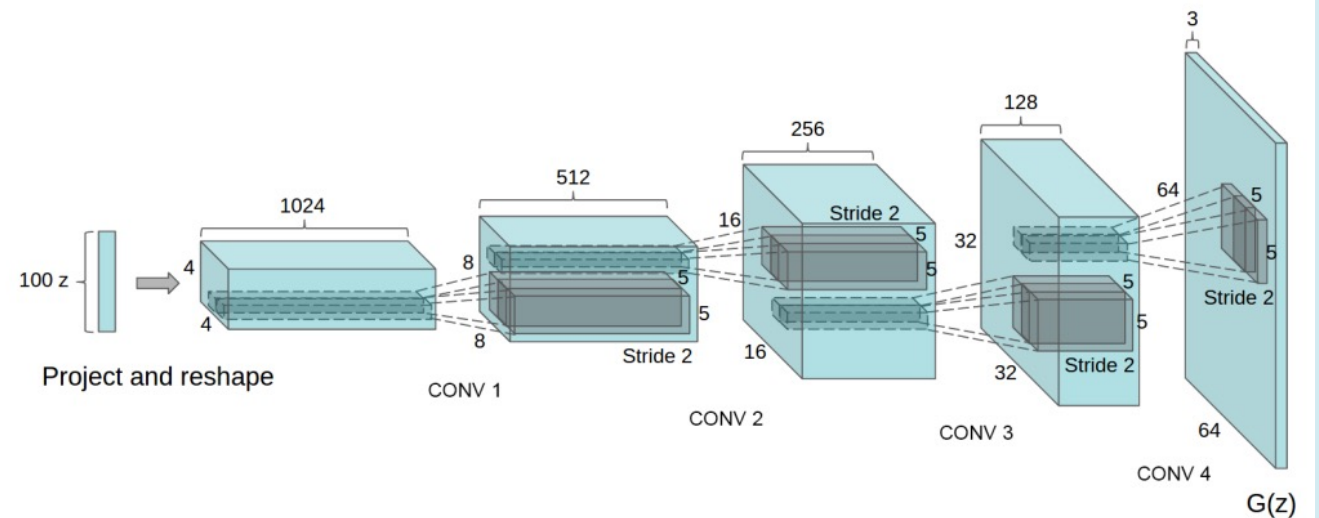
Generator Model

1) the Generator

takes a vector of random noise as input, and generates an image

Example Generator: DCGAN

- An inverted CNN with four **fractionally-strided** convolution layers (not deconvolution)
- These fractional strides grow the size of the image from layer to layer
- The final layer has three channels for red/green/blue



Generative Adversarial Networks (GANs)

A GAN consists of two deterministic neural network models:

1) the Generator

takes a vector of random noise as input,
and generates an image

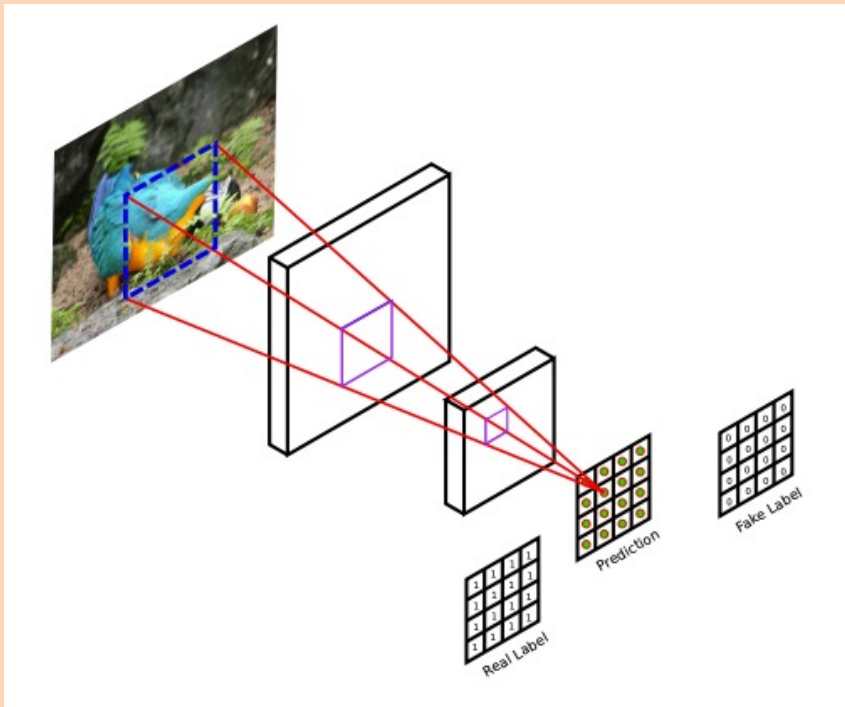
2) the Discriminator

takes in an image classifies whether it is
real (label 1) or fake (label 0)

Discriminator Model

Example Discriminator: PatchGAN

- Convolutional neural network
- Looks at each patch of the image and tries to predict whether it is real or fake
- Helps avoid producing blurry images



2) the Discriminator

takes in an image classifies whether it is real (label 1) or fake (label 0)

Generative Adversarial Networks (GANs)

A GAN consists of two deterministic neural network models:

1) the Generator

takes a vector of random noise as input, and generates an image

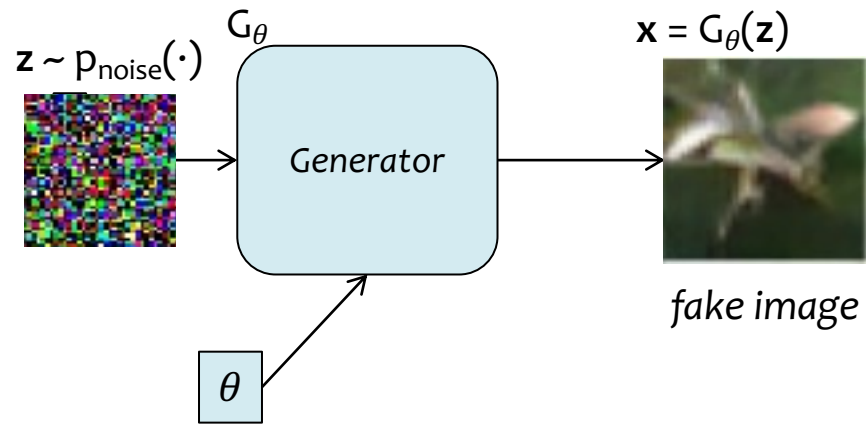
2) the Discriminator

takes in an image classifies whether it is real (label 1) or fake (label 0)

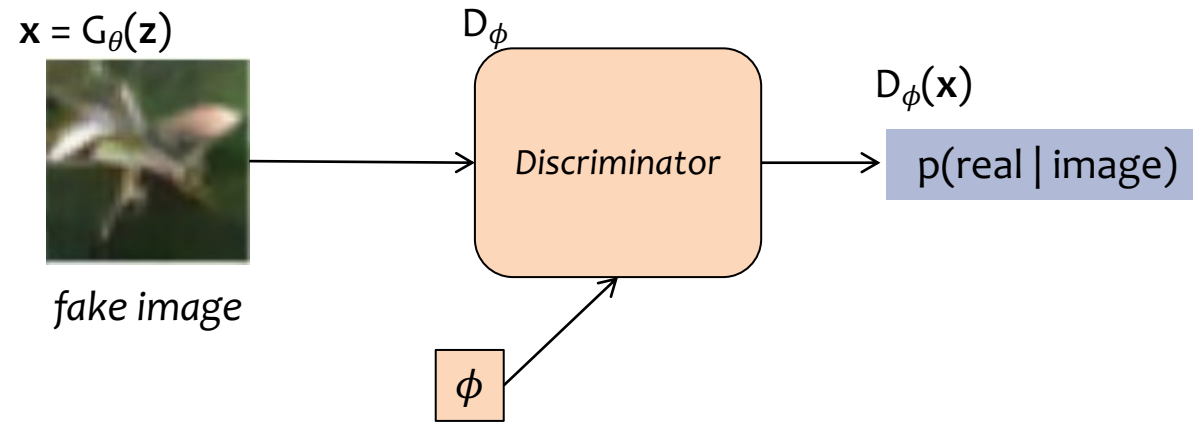
In training, the GAN plays a two player minimax game:

1. the Generator tries to create realistic images to fool the Discriminator into thinking they are real
2. the Discriminator tries to identify the real images from the fake

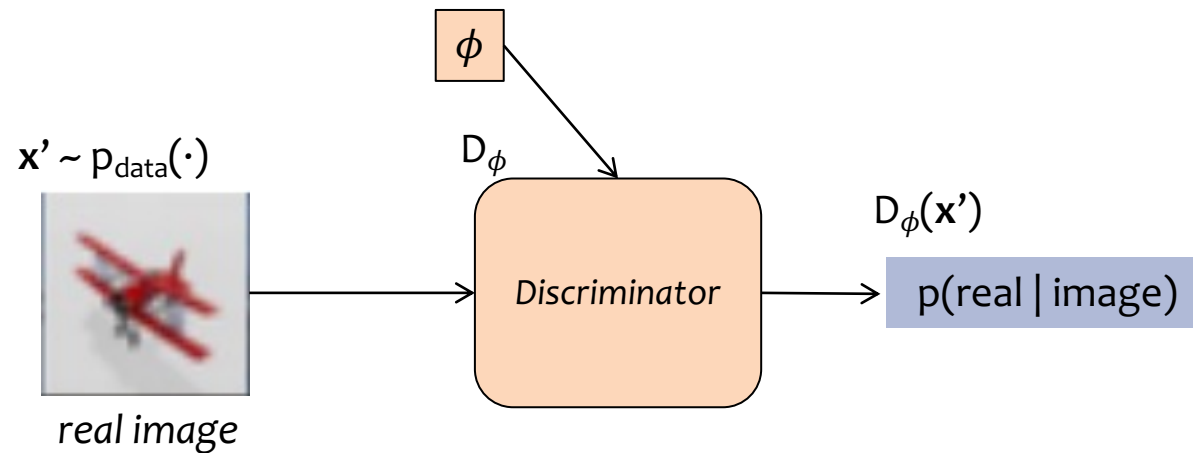
Generative Adversarial Networks (GANs)



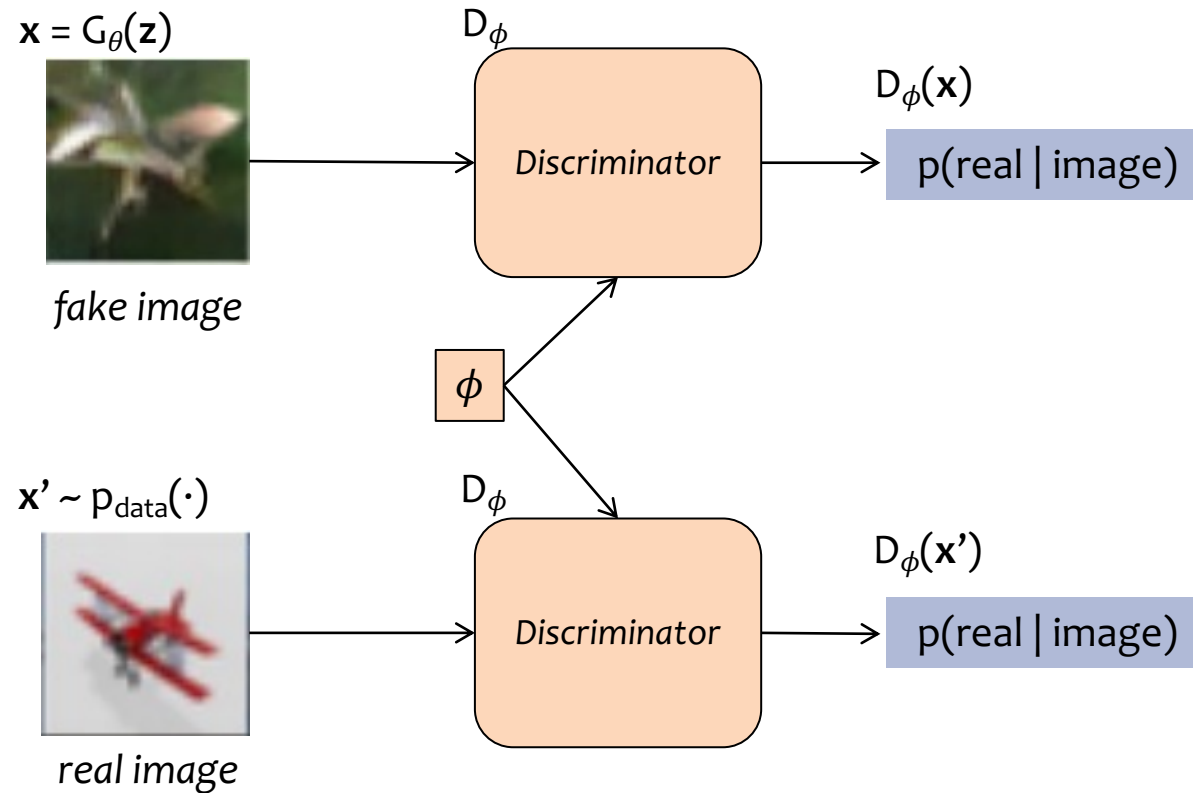
Generative Adversarial Networks (GANs)



Generative Adversarial Networks (GANs)



Generative Adversarial Networks (GANs)



LEARNING FOR GANS

Generative Adversarial Networks (GANs)

A GAN consists of two deterministic neural network models:

1) the Generator

takes a vector of random noise as input, and generates an image

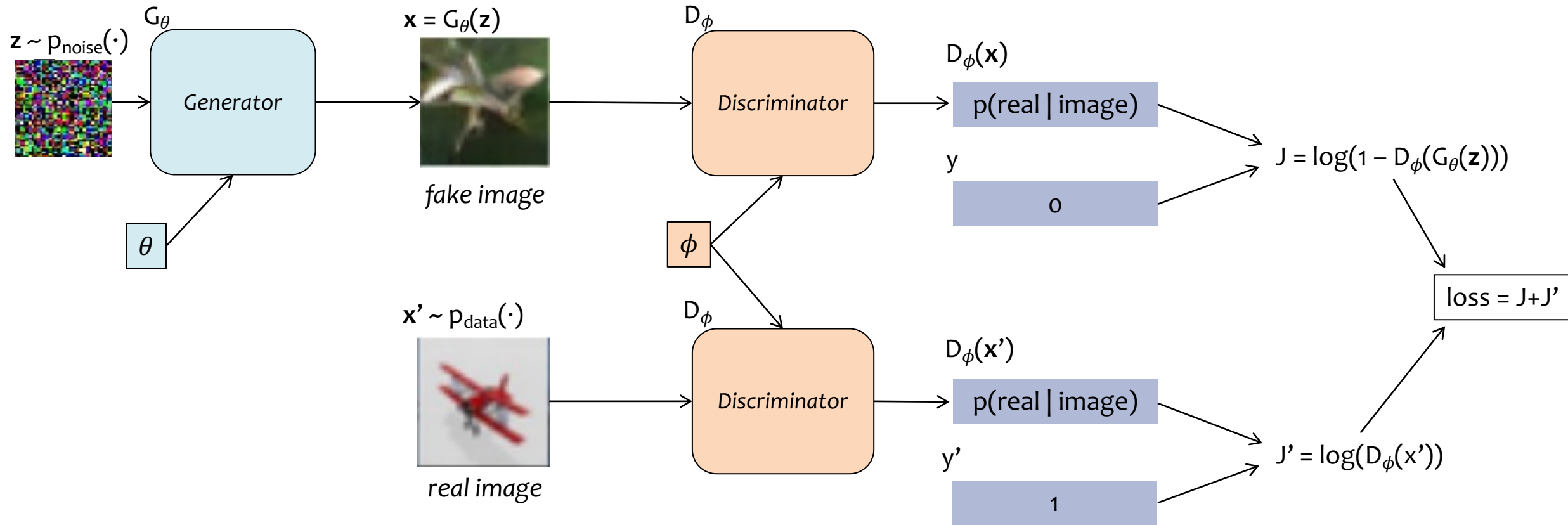
2) the Discriminator

takes in an image classifies whether it is real (label 1) or fake (label 0)

In training, the GAN plays a two player minimax game:

1. the Generator tries to create realistic images to fool the Discriminator into thinking they are real
2. the Discriminator tries to identify the real images from the fake

Generative Adversarial Networks (GANs)



Generative Adversarial Networks (GANs)

$$\max_{\phi} J(\theta, \phi)$$

$$\max_{\phi} \log \left(D_{\phi}(\mathbf{x}^{(i)}) \right) + \log \left(1 - D_{\phi}(G_{\theta}(\mathbf{z}^{(i)})) \right)$$

$$\min_{\theta} \log \left(1 - D_{\phi}(G_{\theta}(\mathbf{z}^{(i)})) \right)$$

$$\min_{\theta} J(\theta, \phi)$$

The discriminator is trying to maximize the likelihood of a binary classifier with labels {real = 1, fake = 0}, on the fixed output of the generator

The generator is trying to minimize the likelihood of its generated (fake) image being classified as fake, according to a fixed discriminator

In training, the GAN plays a two player minimax game:

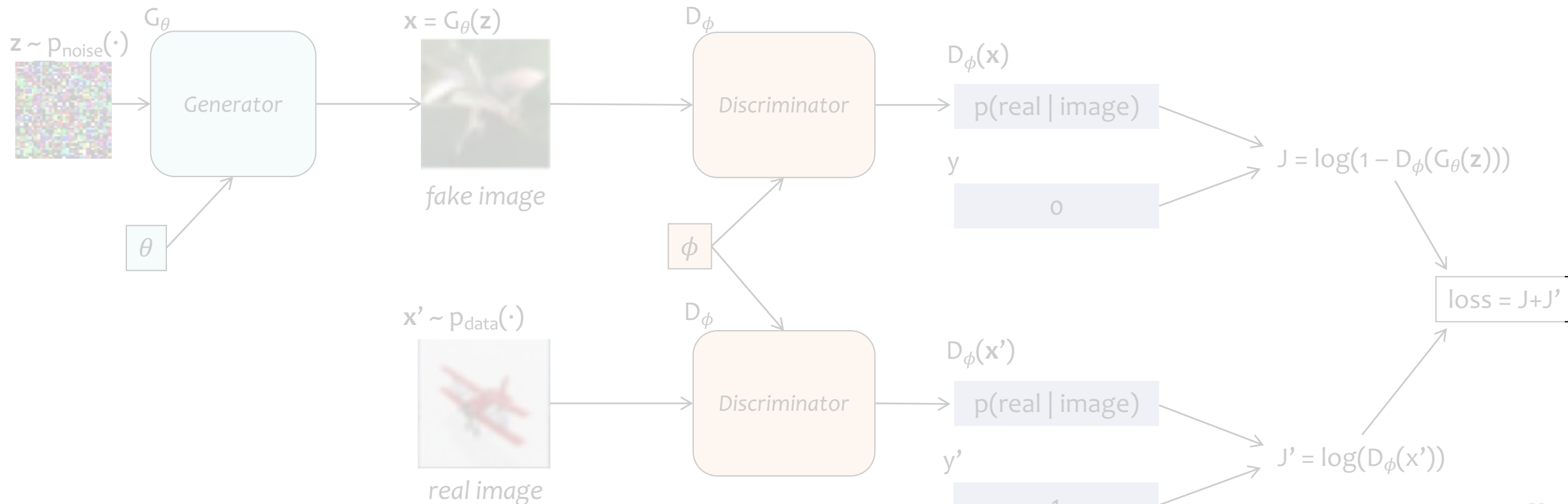
1. the Generator tries to create realistic images to fool the Discriminator into thinking they are real
2. the Discriminator tries to identify the real images from the fake

Learning a GAN

- Objective function is a simple differentiable function
- We chose G and D to be differentiable neural networks

Training alternates between:

- Keep G_θ fixed and backprop through D_ϕ
- Keep D_ϕ fixed and backprop through G_θ

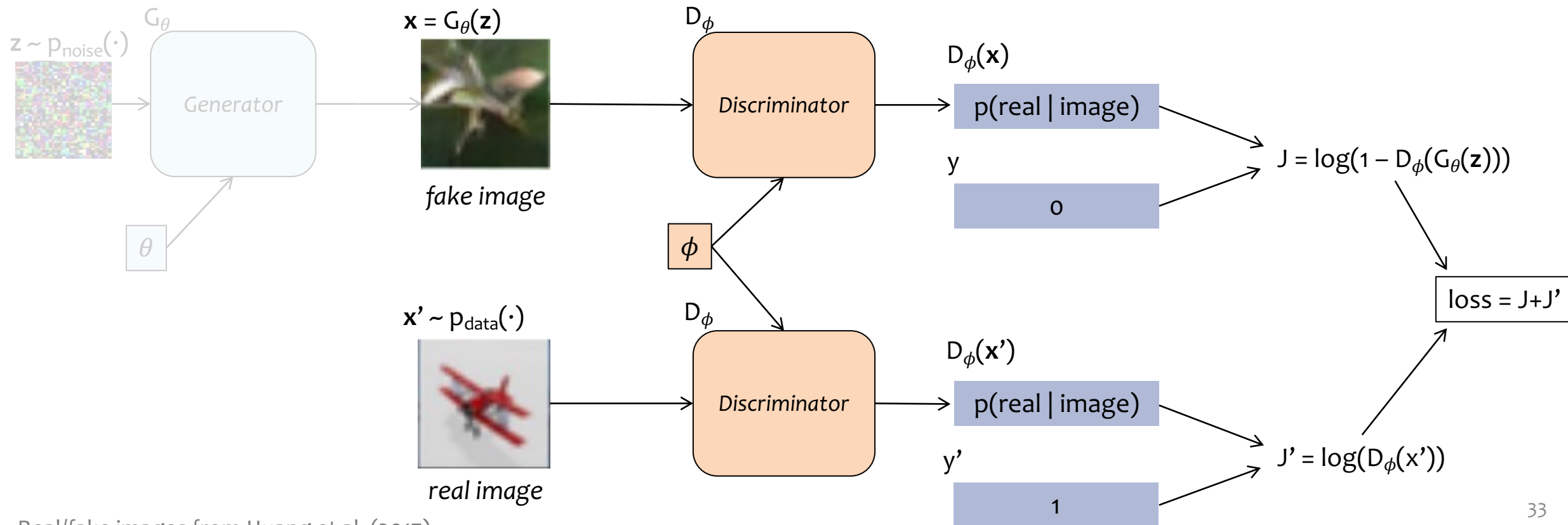


Learning a GAN

- Objective function is a simple differentiable function
- We chose G and D to be differentiable neural networks

Training alternates between:

- **Keep G_θ fixed and backprop through D_ϕ**
- Keep D_ϕ fixed and backprop through G_θ

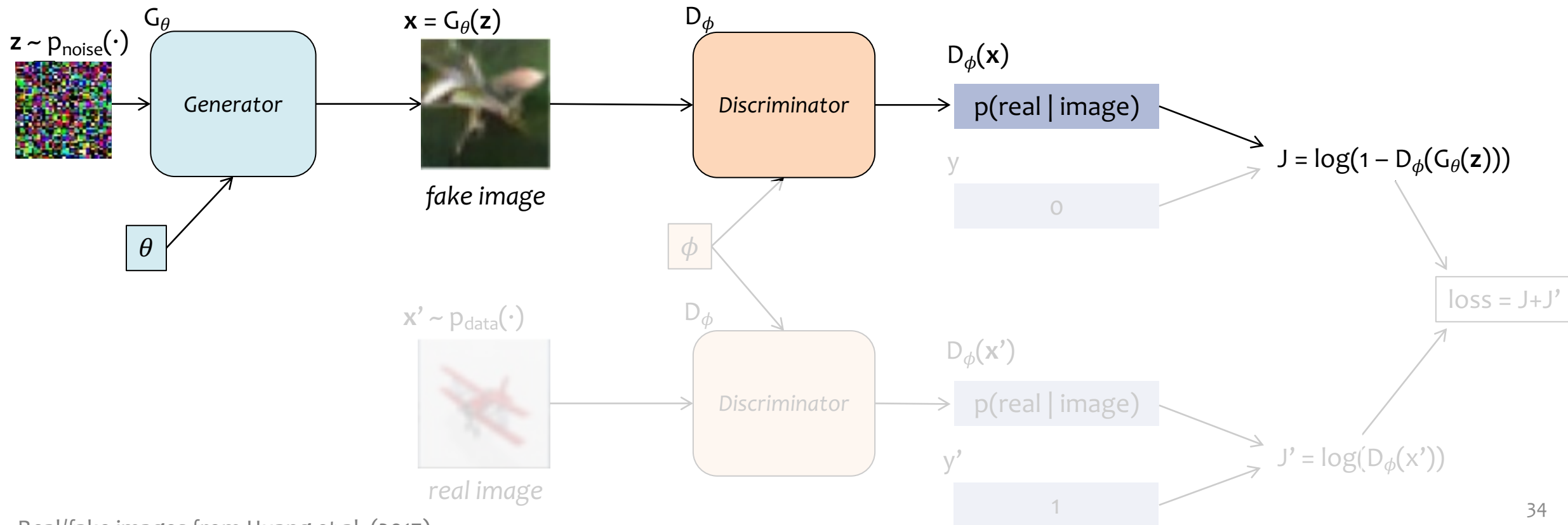


Learning a GAN

- Objective function is a simple differentiable function
- We chose G and D to be differentiable neural networks

Training alternates between:

- Keep G_θ fixed and backprop through D_ϕ
- **Keep D_ϕ fixed and backprop through G_θ**

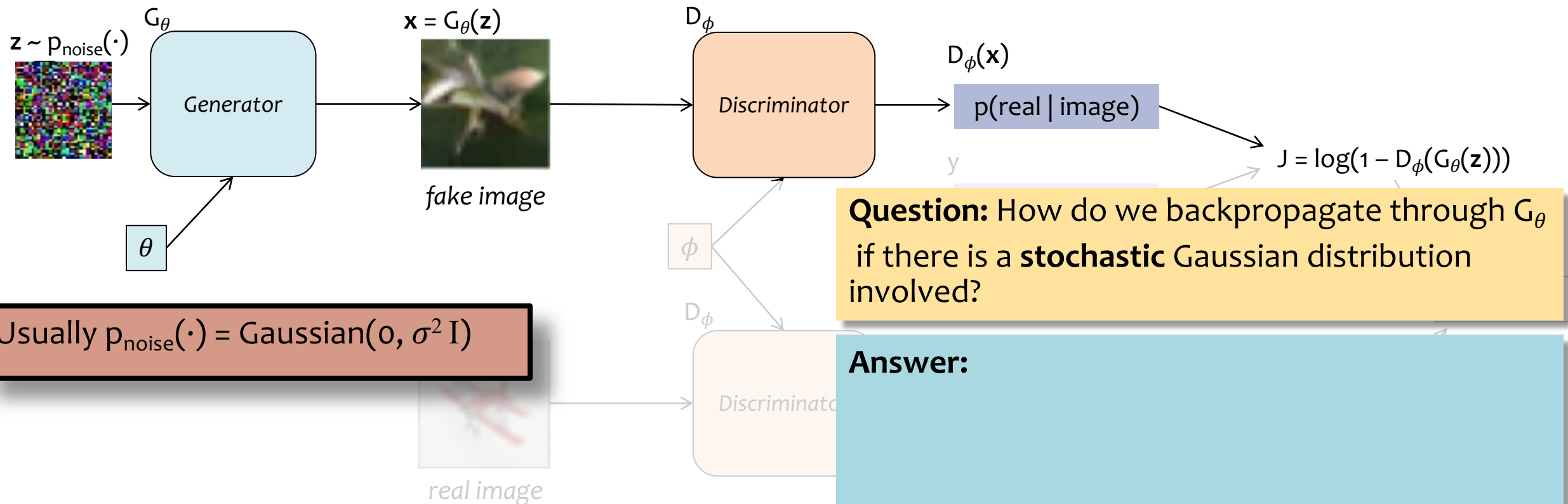


Learning a GAN

- Objective function is a simple differentiable function
- We chose G and D to be differentiable neural networks

Training alternates between:

- Keep G_θ fixed and backprop through D_ϕ
- **Keep D_ϕ fixed and backprop through G_θ**



Learning a GAN

- Training data consists of a collection of m unlabeled images $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}$
- Optimization is similar to block coordinate descent
- But instead of exactly solving the min/max problem, we take a step of mini-batch SGD

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k , is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(m)}\}$ from noise prior $p_g(\mathbf{z})$.
- Sample minibatch of m examples $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\mathbf{x})$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(\mathbf{x}^{(i)}) + \log (1 - D(G(\mathbf{z}^{(i)}))) \right].$$

end for

- Sample minibatch of m noise samples $\{\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(m)}\}$ from noise prior $p_g(\mathbf{z})$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(\mathbf{z}^{(i)}))).$$

end for

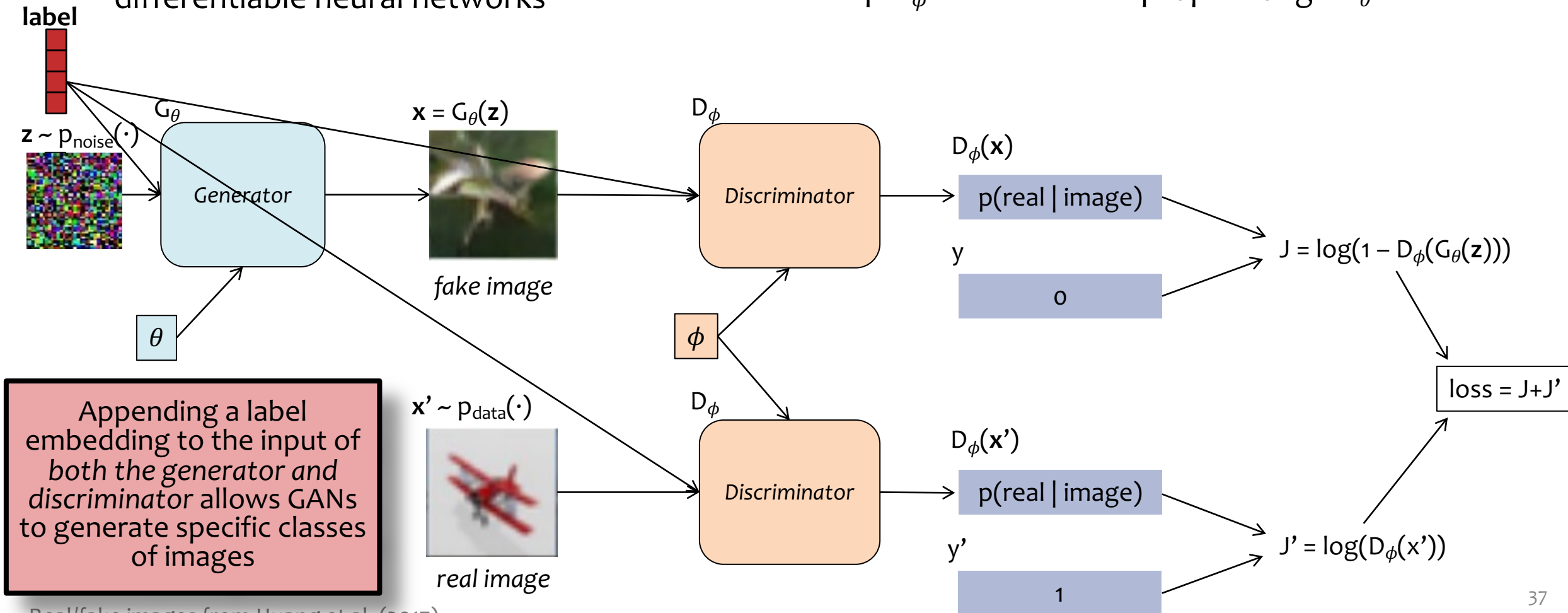
The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

Class-conditional GANs

- Objective function is a simple differentiable function
- We chose G and D to be differentiable neural networks

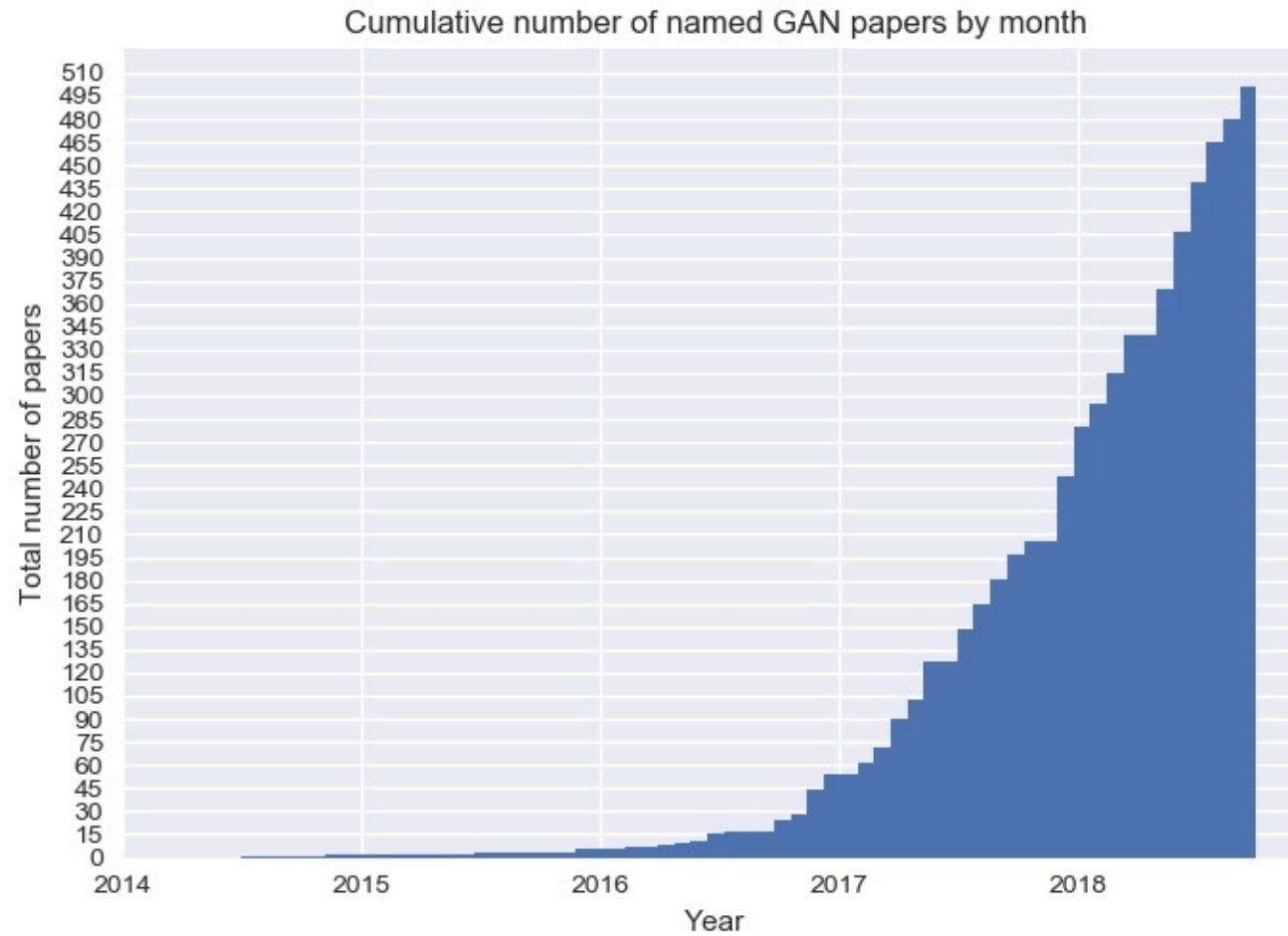
Training alternates between:

- Keep G_θ fixed and backprop through D_ϕ
- Keep D_ϕ fixed and backprop through G_θ

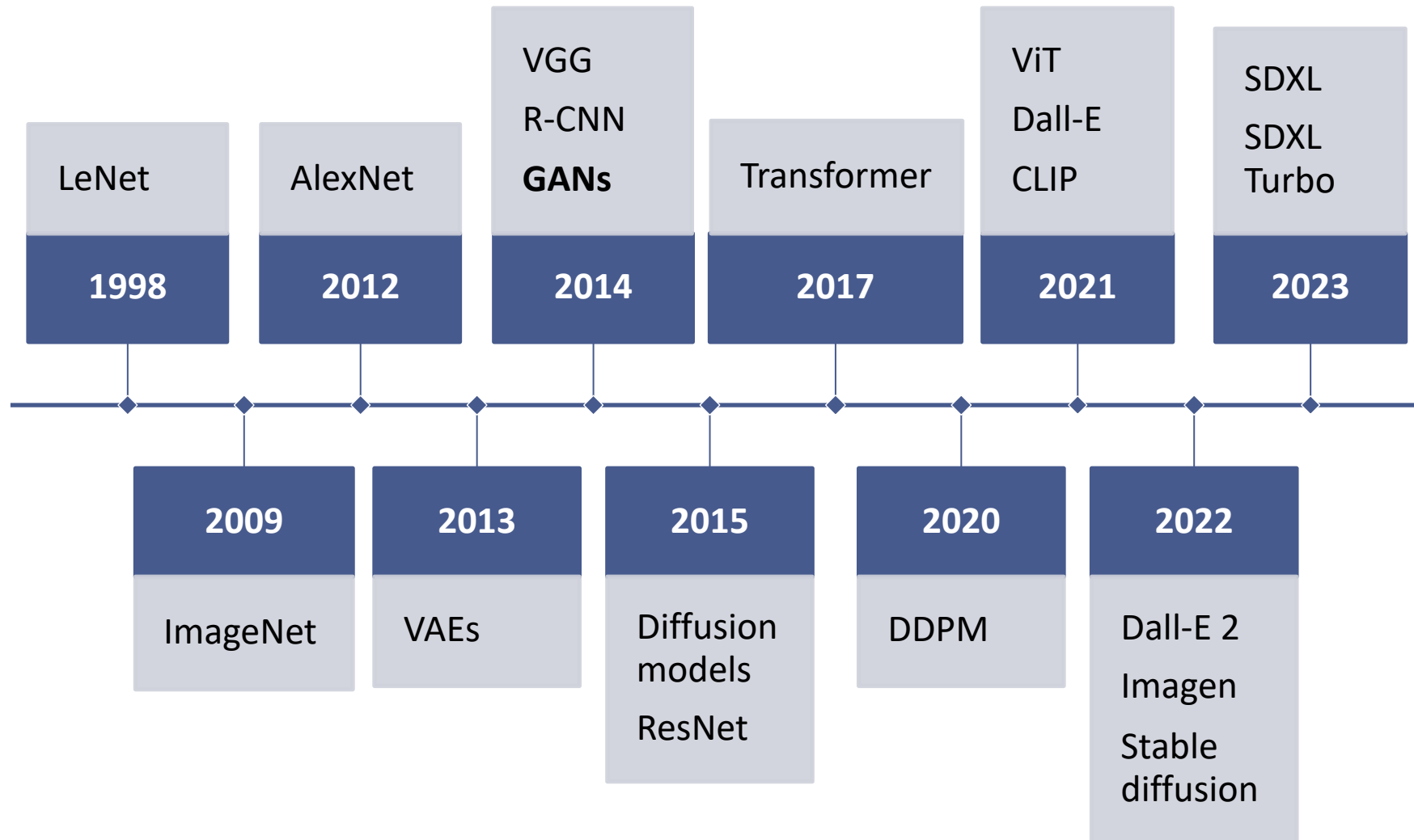


SCALING UP THE MODEL SIZE

GANs Everywhere!



Recall: Computer Vision Timeline



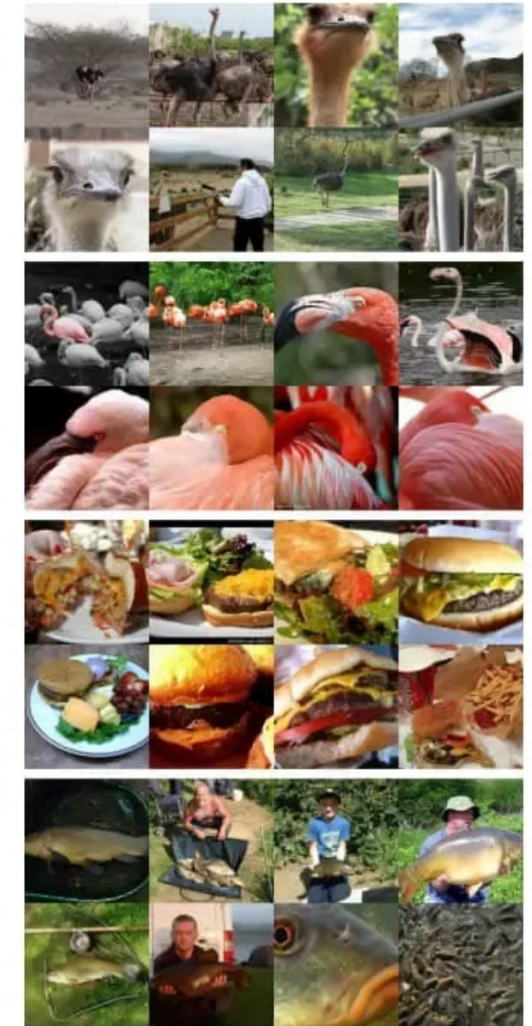
GANs vs. Diffusion



GAN generated
images



Training
images



Diffusion generated
images

Scaling Up the Model Size

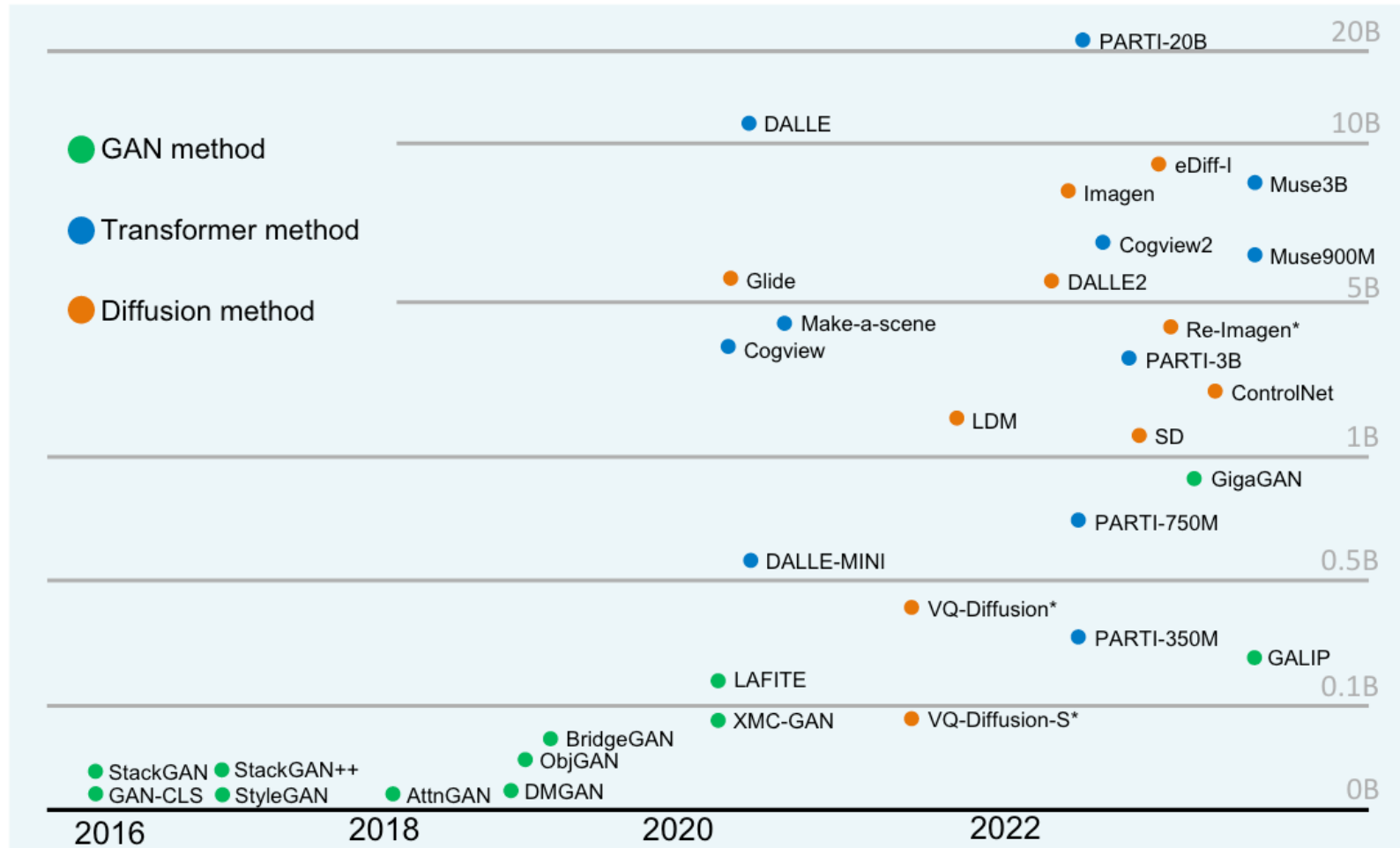


Fig. 5. Timeline of TTI model development, where green dots are GAN TTI models, blue dots are autoregressive Transformers and orange dots are Diffusion TTI models. Models are separated by their parameter, which are in general counted for all their components. Models with asterisk are calculated without the involvement of their text encoders.

Scaling Up the Model Size

Prompt: A portrait photo of a kangaroo wearing an orange hoodie and blue sunglasses standing on the grass in front of the Sydney Opera House holding a sign on the chest that says Welcome Friends!

Parti with different model sizes

350M



750M



3B



20B



Watermarking & Attribution

- **Watermarking**

- A digital watermark allows one to identify when an image has been created by a model
- Most methods for image generation (GANs, VAEs, stable diffusion) can be augmented with watermarking

- **Fake-image Detection**

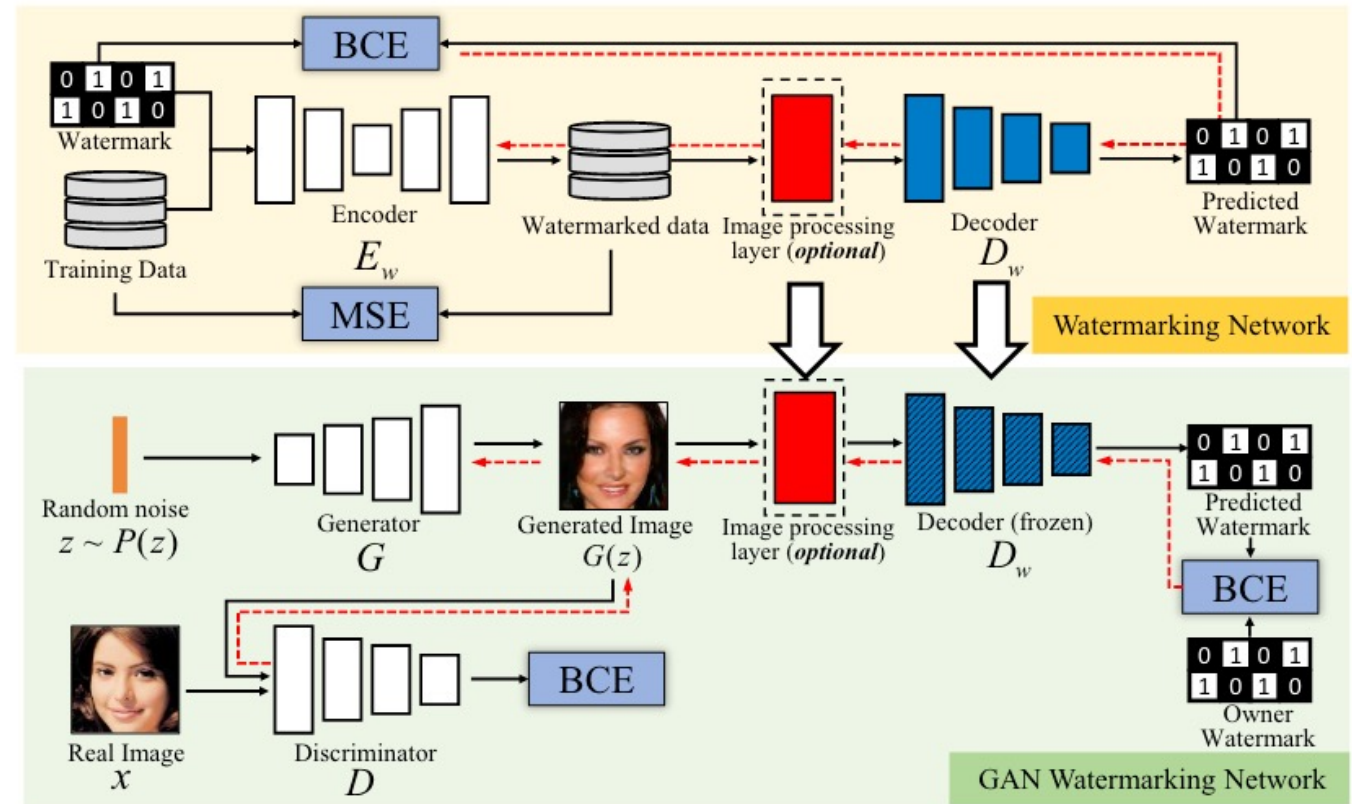
- Goal: identify fakes even without a watermark

- **Model Attribution**

- Identify which generative model created an image (e.g. Dalle-2 vs. SDXL)
- Very successful (natural watermarks)

- **Image Attribution**

- Goal: identify the source images that led to the generation of a new image
- Extremely challenging



Societal Impacts of Image Generation

Pros

- New tools for artists
- Faster creation of memes

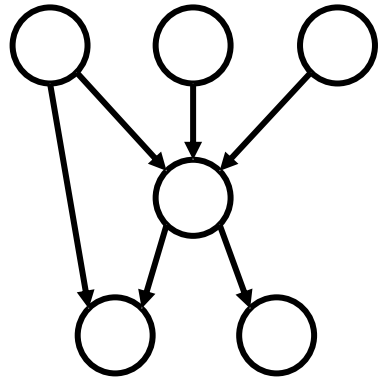
Cons

- Copyright infringement / loss of work for artists
- Societal decrease in creativity
- Potential to create dehumanizing content
- Fake news / false realities / increased difficulty of fact checking
- Not rooted in reality

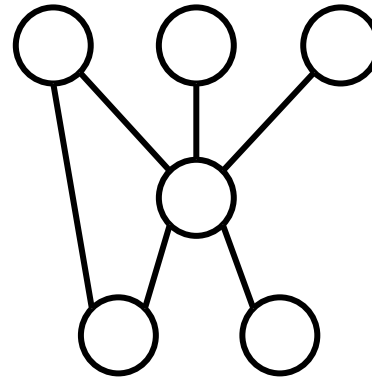
DIRECTED GRAPHICAL MODEL

Three Types of Graphical Models

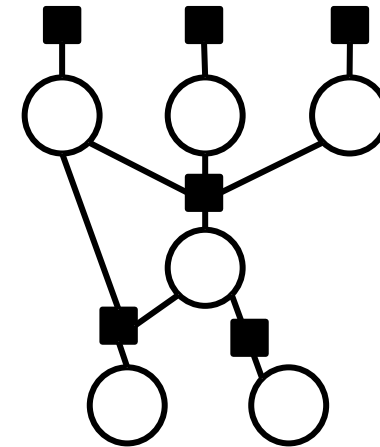
Directed Graphical Model



Undirected Graphical Model

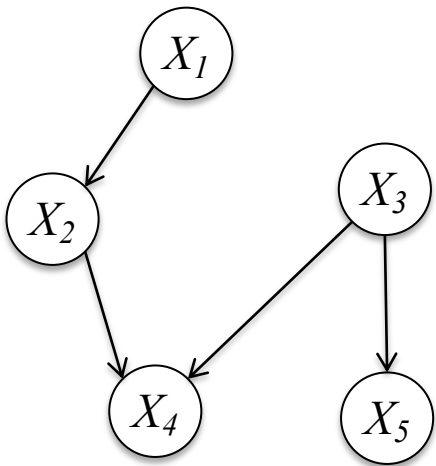


Factor Graph



Directed Graphical Model

Example



$$\begin{aligned} P(X_1, X_2, X_3, X_4, X_5) = \\ P(X_5 \mid X_3)P(X_4 \mid X_2, X_3) \\ P(X_3)P(X_2 \mid X_1)P(X_1) \end{aligned}$$

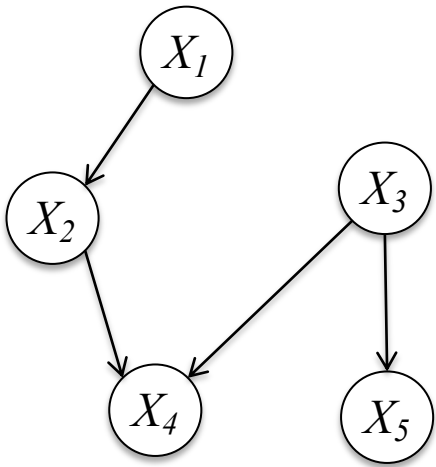
Definition

- A directed graphical model (aka. Bayesian network) is a **directed acyclic graph** that represents the conditional independencies of a set of variables $X_1, \dots, X_T$
- Each **node** is variable X_t and each **edge** implies a directional influence between a pair of variables
- The DGM factorizes the joint distribution over the variables as a product of conditional probabilities:

$$P(X_1, \dots, X_T) = \prod_{t=1}^T P(X_t \mid \text{parents}(X_t))$$

Directed Graphical Model

Example



$$p(x_1, x_2, x_3, x_4, x_5) =$$

$$p(x_5 \mid x_3)p(x_4 \mid x_2, x_3)$$

$$p(x_3)p(x_2 \mid x_1)p(x_1)$$

the graph (*qualitative specification*) could be:

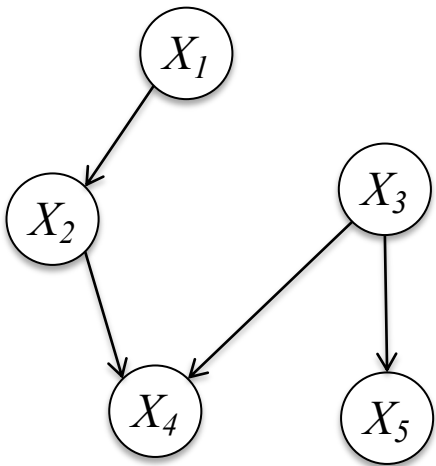
- specified using domain expertise about causal relationships
- learned from data
- chosen because of nice computational properties

the conditional probabilities (*quantitative specification*) is:

- depends on the types of variables involved
- typically learned from data

Directed Graphical Model

Example



$$p(x_1, x_2, x_3, x_4, x_5) =$$

$$p(x_5 | x_3) p(x_4 | x_2, x_3)$$

$$p(x_3) p(x_2 | x_1) p(x_1)$$

Quantitative Specification

x_1	$p(x_1)$
red	0.4
blue	0.6

x_2	x_1	$p(x_2 x_1)$
red	red	0.7
blue	red	0.3
red	blue	0.5
blue	blue	0.5

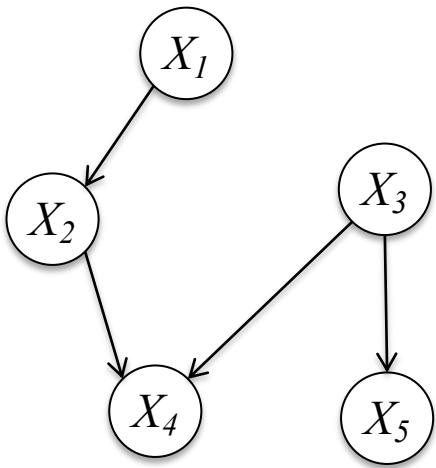
x_3	$p(x_3)$
red	0.6
blue	0.4

x_4	x_2	x_3	$p(x_4 x_2, x_3)$
red	red	red	0.5
blue	red	red	0.5
red	red	blue	0.8
blue	red	blue	0.2
red	blue	red	0.6
blue	blue	red	0.4
red	blue	blue	0.3
blue	blue	blue	0.7

x_5	x_3	$p(x_5 x_3)$
red	red	0.7
blue	red	0.3
red	blue	0.4
blue	blue	0.6

Directed Graphical Model

Example



$$p(x_1, x_2, x_3, x_4, x_5) =$$

$$p(x_5 \mid x_3)p(x_4 \mid x_2, x_3)$$

$$p(x_3)p(x_2 \mid x_1)p(x_1)$$

Quantitative Specification

If each variable is continuous, we could define Gaussian conditional distributions

$$x_1 \sim \mathcal{N}(\mu_1, \sigma_1)$$

$$x_2 \sim \mathcal{N}(x_1, \sigma_2)$$

$$x_3 \sim \mathcal{N}(\mu_3, \sigma_3)$$

$$x_4 \sim \mathcal{N}(x_2 + x_3, \sigma_4)$$

$$x_5 \sim \mathcal{N}(x_3, \sigma_5)$$

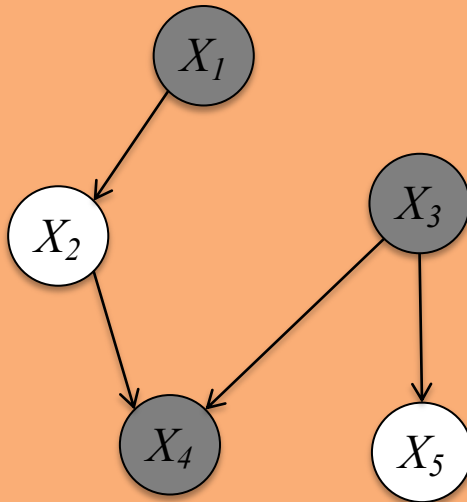
$$P(x_2 \mid x_1)$$

Observed Variables

- In a graphical model, **shaded nodes** are “**observed**”, i.e. their values are given

Example:

$$P(X_2, X_5 \mid X_1 = 0, X_3 = 1, X_4 = 1)$$



MARKOV MODEL

Markov Model

1. **1st-order Markov assumption:**

for a sequence of random variables, the probability distribution over x_t random variables is conditionally independent of $x_1, \dots, x_{t-2}$ given x_{t-1}

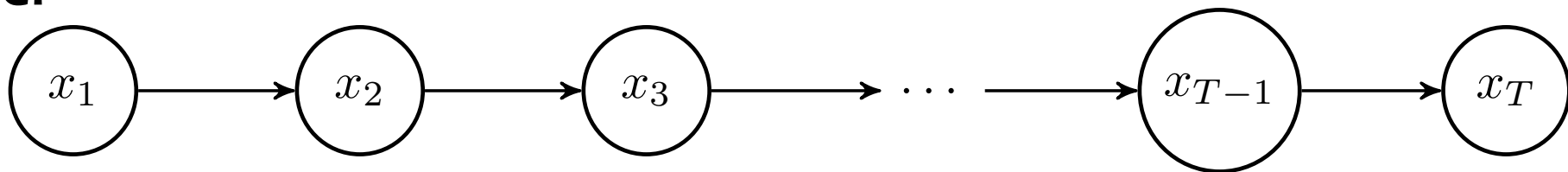
$$p(x_t \mid x_1, \dots, x_{t-1}) = p(x_t \mid x_{t-1})$$

2. **1st-order Markov model:**

defines a joint distribution over a sequence of variables using a Markov assumption

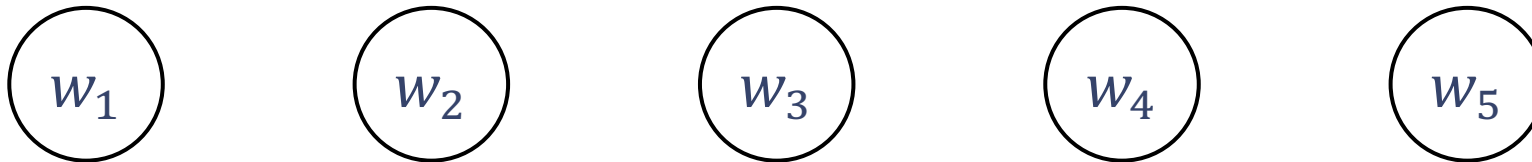
$$p(x_1, \dots, x_T) = p(x_1) \prod_{t=2}^T p(x_t \mid x_{t-1})$$

3. We can represent the Markov model as a **directed graphical model**



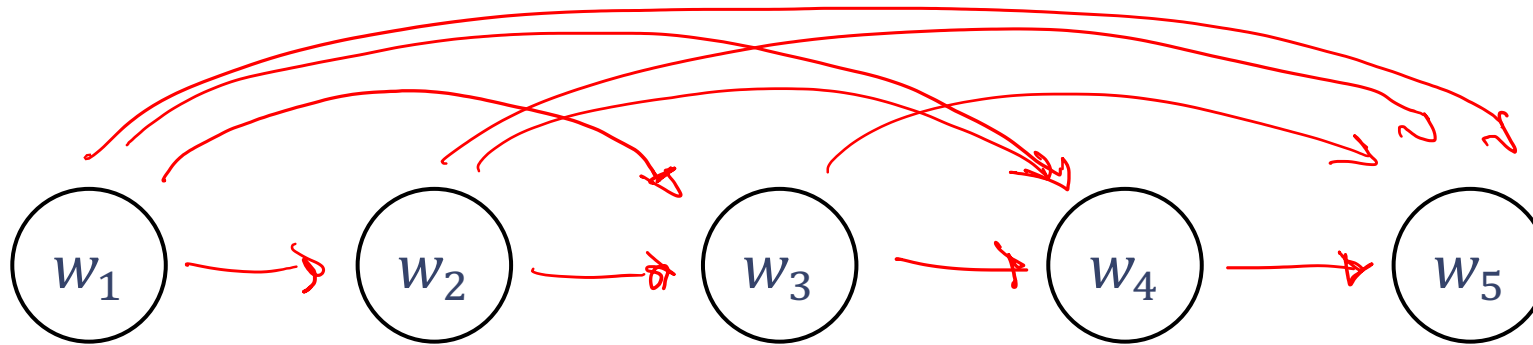
In-class Exercise: RNN-LM as a DGM

Given a five-word sequence, $[w_1, w_2, w_3, w_4, w_5]$, how could we represent the implied probability distributions of an RNN-LM as a directed graphical model?



In-class Exercise: RNN-LM as a DGM

Given a five-word sequence, $[w_1, w_2, w_3, w_4, w_5]$, how could we represent the implied probability distributions of an RNN-LM as a directed graphical model?

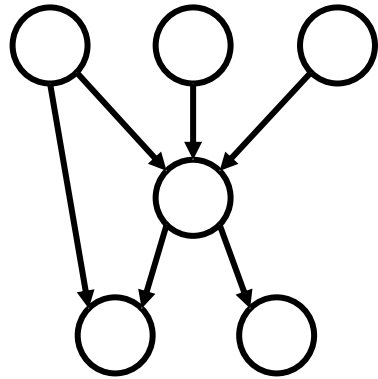


$$p(w_1, w_2, w_3, w_4, w_5) = p(w_1) p(w_2 \mid w_1) p(w_3 \mid w_1, w_2) p(w_4 \mid w_1, w_2, w_3) p(w_5 \mid w_1, w_2, w_3, w_4)$$

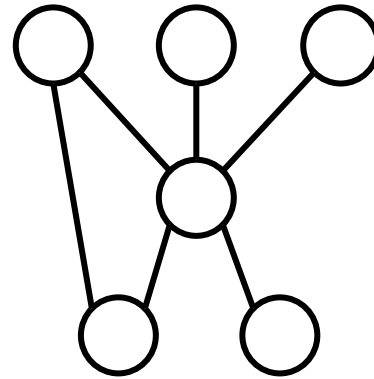
UNDIRECTED GRAPHICAL MODELS

Three Types of Graphical Models

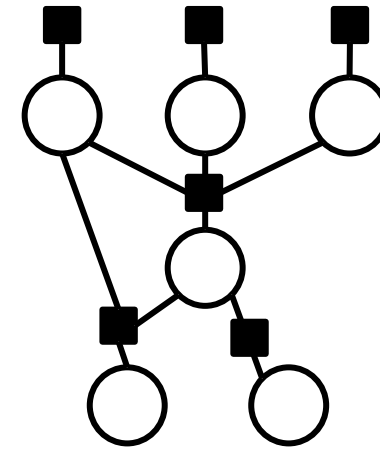
Directed Graphical Model



Undirected Graphical Model



Factor Graph



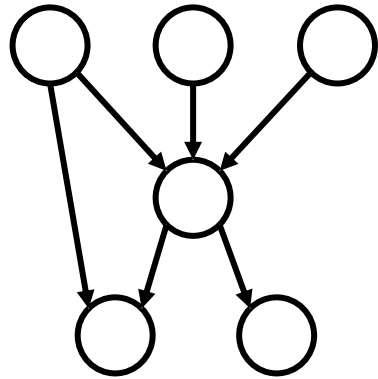
Undirected Graphical Models

Representation of both directed and undirected graphical models

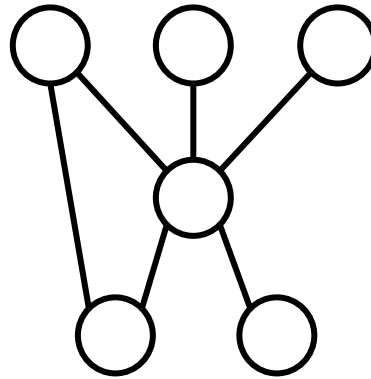
FACTOR GRAPHS

Three Types of Graphical Models

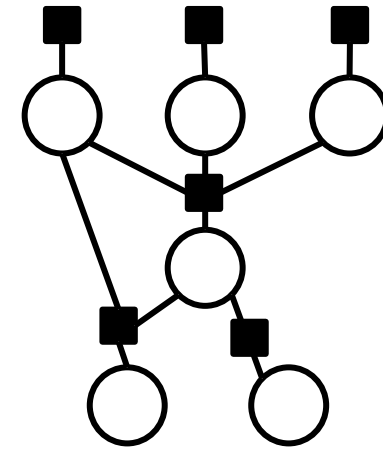
Directed Graphical Model



Undirected Graphical Model



Factor Graph

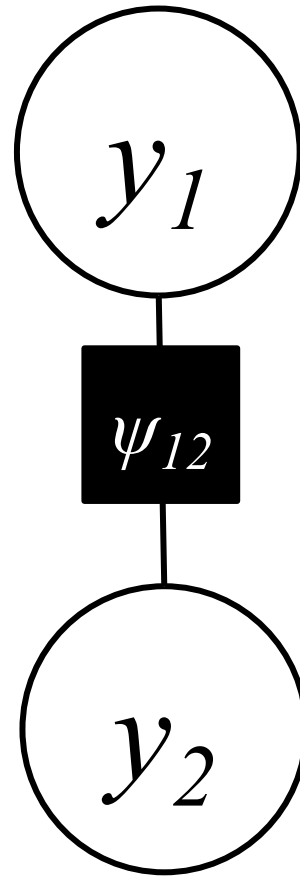


Factor Graphs

Factor Graph

(bipartite graph)

- variables (circles)
- factors (squares)

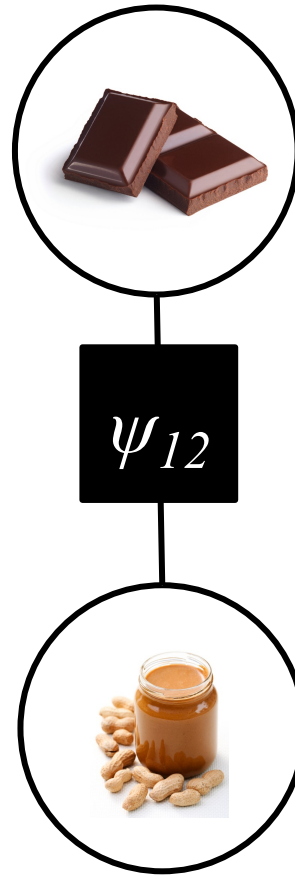


Factor Graphs

Factor Graph

(bipartite graph)

- variables (circles)
- factors (squares)



Each **random variable** can be assigned a **value**

The collection of values for all the random variables is called an **assignment**.

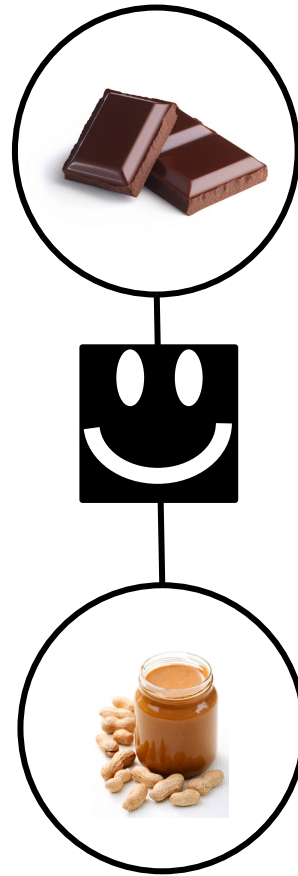
Factor Graphs

Factor Graph

(bipartite graph)

- variables (circles)
- factors (squares)

Factors have
local opinions
about the
assignments of
their
neighboring
variables



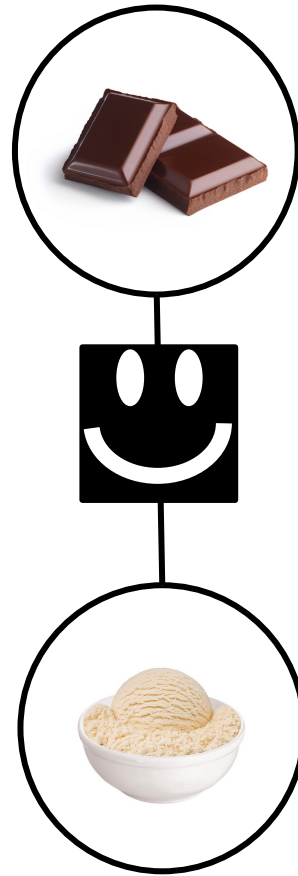
Factor Graphs

Factor Graph

(bipartite graph)

- variables (circles)
- factors (squares)

Factors have
local opinions
about the
assignments of
their
neighboring
variables



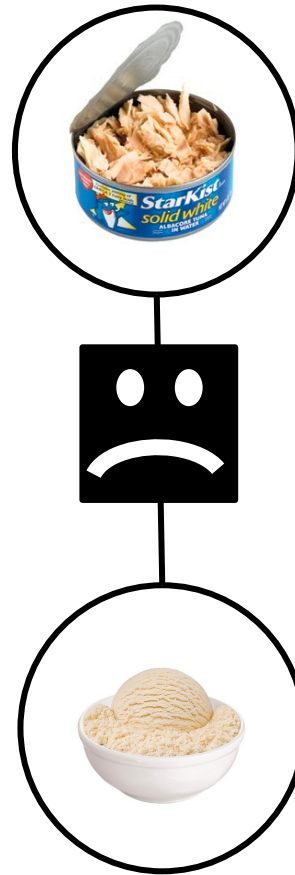
Factor Graphs

Factor Graph

(bipartite graph)

- variables (circles)
- factors (squares)

Factors have
local opinions
about the
assignments of
their
neighboring
variables

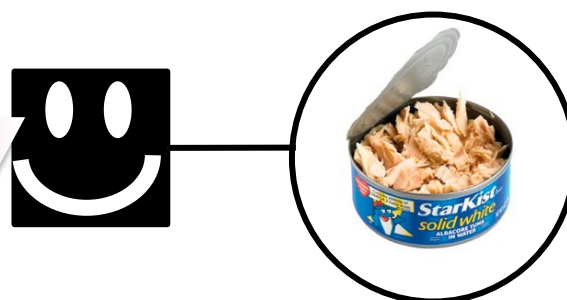


Factor Graphs

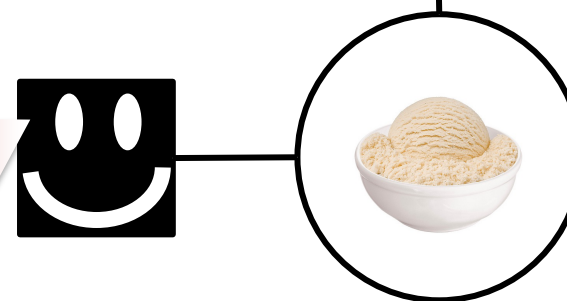
$$P(\text{tuna, ice cream}) = ?$$

Those opinions are
expressed through
potential tables

chocolate	0.1
peanut butter	5
ice cream	1
tuna	6
...	



chocolate	4
peanut butter	8
ice cream	7
tuna	3
...	



	chocolate	peanut butter	Ice cream	tuna	...
chocolate	2	9	7	0.1	
peanut butter	4	2	3	0.2	
ice cream	7	3	2	0.1	
tuna	0.1	0.2	0.1	2	
...					

Factor Graphs

$$P(\text{tuna, ice cream}) = \frac{1}{Z} (6 * 7 * 0.1)$$

chocolate	0.1
peanut butter	5
ice cream	1
tuna	6
...	



chocolate	4
peanut butter	8
ice cream	7
tuna	3
...	

*Uh-oh! The probabilities of
the various assignments sum
up to $Z > 1$.
So divide them all by Z .*

	cho	peanu	Ice	tuna	...
chocolate	2	9	7	0.1	
peanut butter	4	2	3	0.2	
ice cream	7	3	2	0.1	
tuna	0.1	0.2	0.1	2	
...					

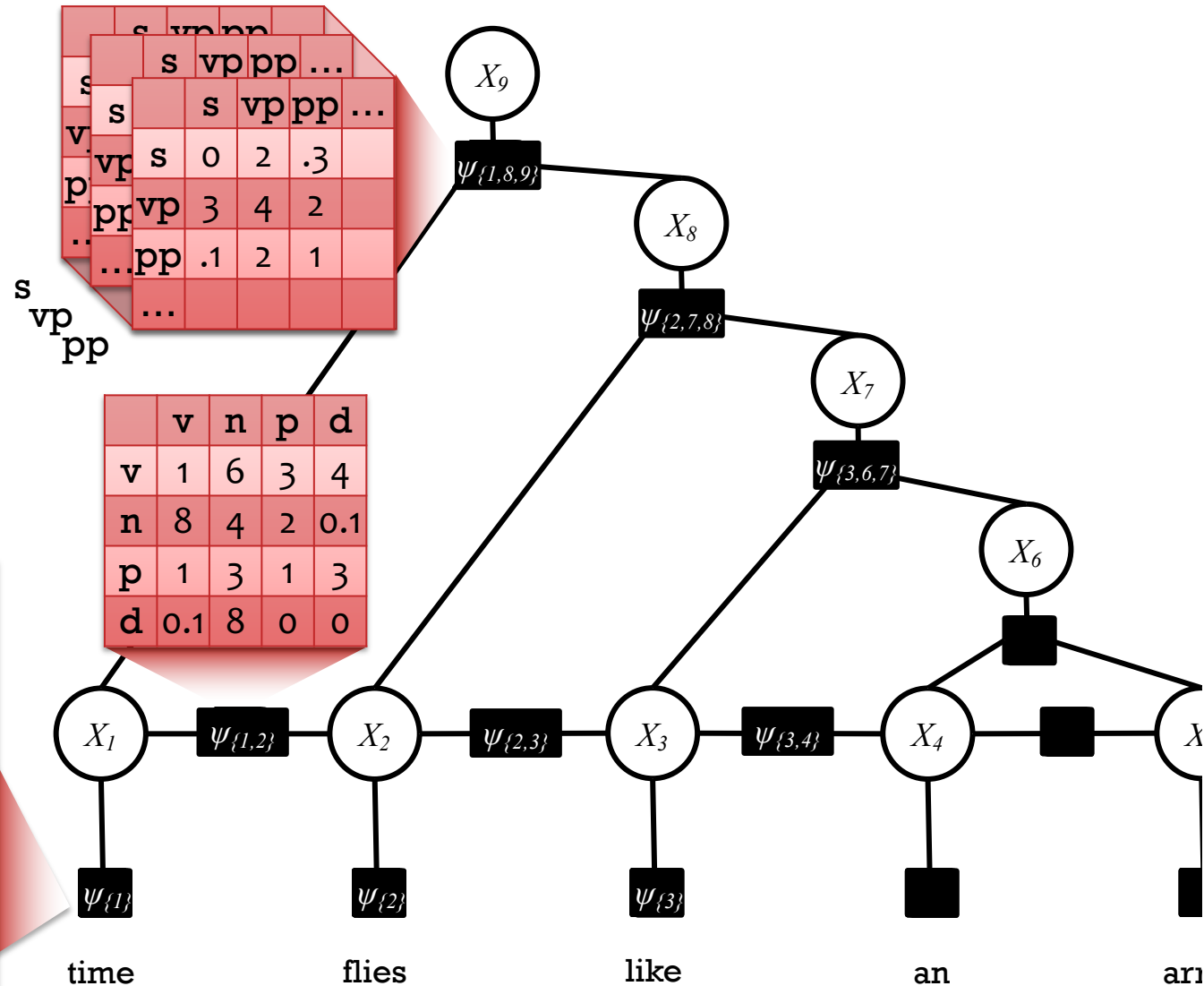
The combined potential tables of all factors
defines the probability of an assignment

Factors are Tensors

- Factors must contain **non-negative** values -- this ensures we have a valid probability distribution
- We also sometimes refer to factors as **potential functions** or **potentials** (like UGMs)

Joint Distribution

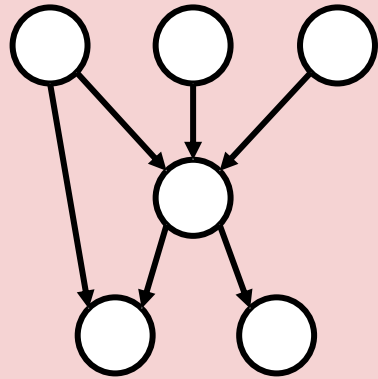
$$p(\mathbf{x}) = \frac{1}{Z} \prod_{\alpha} \psi_{\alpha}(\mathbf{x}_{\alpha})$$



Locally Normalized

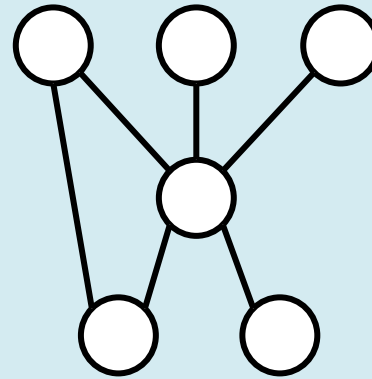
vs. Globally Normalized

Directed Graphical Model

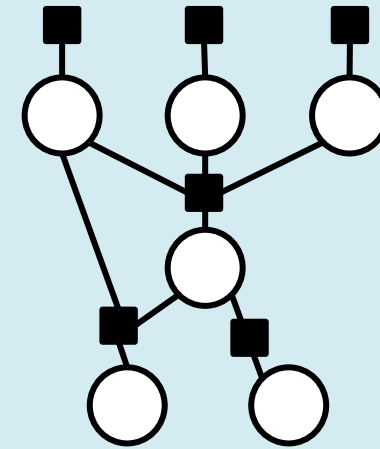


$$P(X_1, \dots, X_T) = \prod_{t=1}^T P(X_t \mid \text{parents}(X_t))$$

Undirected Graphical Model



Factor Graph



$$p(\mathbf{x}) = \frac{1}{Z} \prod_{\alpha} \psi_{\alpha}(\mathbf{x}_{\alpha})$$

Inference

Three Tasks:

1. Marginal Inference (#P-Hard)

Compute marginals of variables and cliques

$$p(x_i) = \sum_{\mathbf{x}': x'_i = x_i} p(\mathbf{x}' \mid \boldsymbol{\theta}) \quad \Bigg| \quad p(\mathbf{x}_C) = \sum_{\mathbf{x}': \mathbf{x}'_C = \mathbf{x}_C} p(\mathbf{x}' \mid \boldsymbol{\theta})$$

2. Partition Function (#P-Hard)

Compute the normalization constant

$$Z(\boldsymbol{\theta}) = \sum_{\mathbf{x}} \prod_{C \in \mathcal{C}} \psi_C(\mathbf{x}_C)$$

3. MAP Inference (NP-Hard)

Compute variable assignment with highest probability

$$\hat{\mathbf{x}} = \operatorname{argmax}_{\mathbf{x}} p(\mathbf{x} \mid \boldsymbol{\theta})$$

UNSUPERVISED LEARNING

Unsupervised Learning

Assumptions:

1. our data comes from some distribution $p^*(\mathbf{x}_o)$
2. we choose a distribution $p_\theta(\mathbf{x}_o)$ for which sampling $\mathbf{x}_o \sim p_\theta(\mathbf{x}_o)$ is tractable

Goal: learn θ s.t. $p_\theta(\mathbf{x}_o) \approx p^*(\mathbf{x}_o)$

Unsupervised Learning

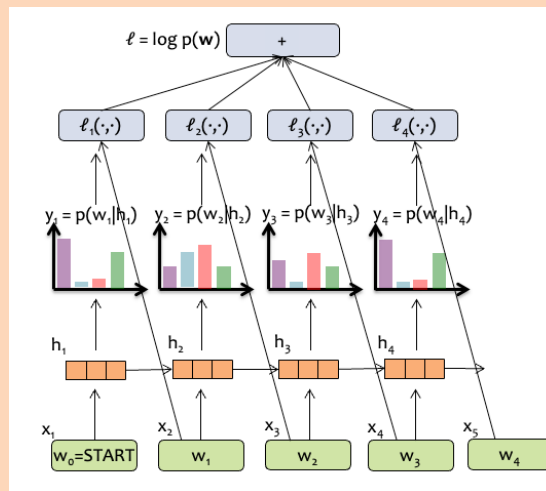
Assumptions:

1. our data comes from some distribution $p^*(\mathbf{x}_0)$
2. we choose a distribution $p_\theta(\mathbf{x}_0)$ for which sampling $\mathbf{x}_0 \sim p_\theta(\mathbf{x}_0)$ is tractable

Goal: learn θ s.t. $p_\theta(\mathbf{x}_0) \approx p^*(\mathbf{x}_0)$

Example: autoregressive LMs

- true $p^*(\mathbf{x}_0)$ is the (human) process that produced text on the web
- choose $p_\theta(\mathbf{x}_0)$ to be an autoregressive language model
 - autoregressive structure means that $p(\mathbf{x}_t | \mathbf{x}_1, \dots, \mathbf{x}_{t-1})$ so ancestral sampling is exact/efficient
- learn by finding
$$\theta \approx \operatorname{argmax}_\theta \log(p_\theta(\mathbf{x}_0))$$
using gradient based updates on
$$\nabla_\theta \log(p_\theta(\mathbf{x}_0))$$



Unsupervised Learning

Assumptions:

1. our data comes from some distribution $p^*(\mathbf{x}_0)$
2. we choose a distribution $p_\theta(\mathbf{x}_0)$ for which sampling $\mathbf{x}_0 \sim p_\theta(\mathbf{x}_0)$ is tractable

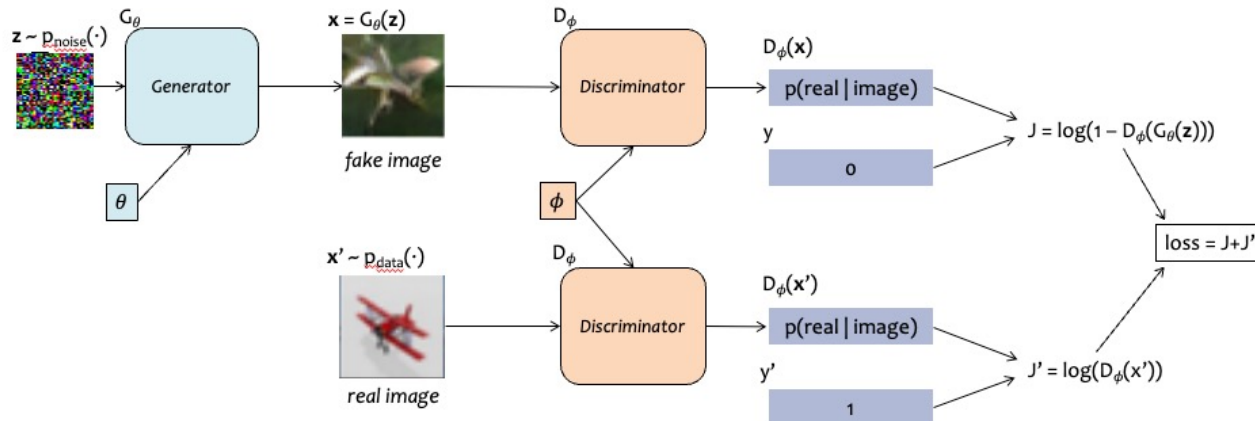
Goal: learn θ s.t. $p_\theta(\mathbf{x}_0) \approx p^*(\mathbf{x}_0)$

Example: GANs

- true $p^*(\mathbf{x}_0)$ is distribution over photos taken and posted to Flickr
- choose $p_\theta(\mathbf{x}_0)$ to be an expressive model (e.g. noise fed into inverted CNN) that can generate images
 - sampling is typically easy:
 $\mathbf{z} \sim N(\mathbf{0}, \mathbf{I})$ and $\mathbf{x}_0 = f_\theta(\mathbf{z})$
- learn by finding $\theta \approx \operatorname{argmax}_\theta \log(p_\theta(\mathbf{x}_0))$?
 - No! Because we can't even compute $\log(p_\theta(\mathbf{x}_0))$ or its gradient
 - Why not? Why not? Even for 1-layer neural networks: *single* image $\mathbf{x}_0$ mapping from noise $\mathbf{z}$ in conditional makes marginal $p(\mathbf{x}_0)$ almost 0 everywhere & difficult to differentiate

$$p_\theta(\mathbf{x}_0) = \int_{\mathbf{z}} p_\theta(\mathbf{x}_0 | \mathbf{z}) p(\mathbf{z}) d\mathbf{z}$$

so optimize a minimax loss instead



Unsupervised Learning

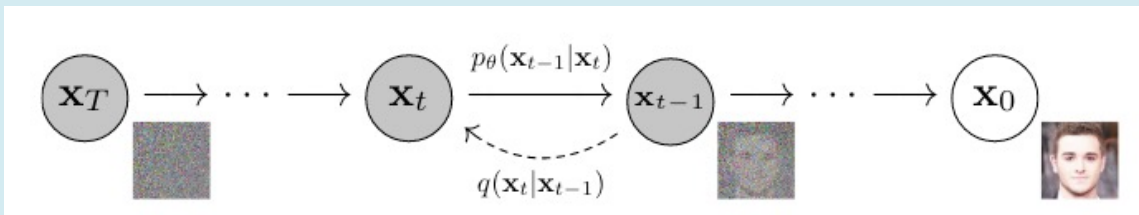
Assumptions:

1. our data comes from some distribution $p^*(\mathbf{x}_0)$
2. we choose a distribution $p_\theta(\mathbf{x}_0)$ for which sampling $\mathbf{x}_0 \sim p_\theta(\mathbf{x}_0)$ is tractable

Goal: learn θ s.t. $p_\theta(\mathbf{x}_0) \approx p^*(\mathbf{x}_0)$

Example: VAEs / Diffusion Models

- true $p^*(\mathbf{x}_0)$ is distribution over photos taken and posted to Flickr
- choose $p_\theta(\mathbf{x}_0)$ to be an expressive model (e.g. noise fed into inverted CNN) that can generate images
 - sampling is will be easy
- learn by finding $\theta \approx \operatorname{argmax}_\theta \log(p_\theta(\mathbf{x}_0))$?
 - Sort of! We can't compute the gradient $\nabla_\theta \log(p_\theta(\mathbf{x}_0))$
 - So we instead optimize a variational lower bound (more on that later)



Latent Variable Models

- For GANs and VAEs, we assume that there are (unknown) **latent variables** which give rise to our observations
- The **vector z** are those latent variables
- After learning a GAN or VAE, we can **interpolate** between images in latent z space



Figure 4: Top rows: Interpolation between a series of 9 random points in Z show that the space learned has smooth transitions, with every image in the space plausibly looking like a bedroom. In the 6th row, you see a room without a window slowly transforming into a room with a giant window. In the 10th row, you see what appears to be a TV slowly being transformed into a window.

DIFFUSION MODELS AND VARIATIONAL AUTOENCODERS (VAES)

Diffusion Models

- Next we will consider (1) **diffusion models** and (2) **variational autoencoders (VAEs)**
 - Although VAEs came first, we're going to dive into diffusion models since they will receive more of our attention
- The steps in defining these models is roughly:
 - Define a probability distribution involving Gaussian noise
 - Use a variational lower bound as an objective function
 - Learn the parameters of the probability distribution by optimizing the objective function
- So what is a variational lower bound?

Diffusion Models

- Next we will consider (1) **diffusion models** and (2) **variational autoencoders (VAEs)**
 - Although VAEs came first, we're going to focus on diffusion models since they will receive more of our attention
- The steps in defining these models is as follows:
 - Define a probability distribution involving a sequence of latent variables
 - Use a variational lower bound as an objective function
 - Learn the parameters of the probability distribution by maximizing the objective function
- So what is a variational lower bound?

The standard presentation of diffusion models requires an understanding of variational inference. (we'll do that in Lecture 9)

Next time (Lecture 7), we'll do an alternate presentation *without* variational inference!